
PROMPT INJECTION DEFENSES IN LARGE LANGUAGE MODELS (LLMs): A MULTI-LAYER DETECTION APPROACH

Adrian Lopez
Department of Computer Science
Texas A&M University–San Antonio
San Antonio, TX
Alope0321@jaguar.tamu.edu

Mallory A. Sorola
Department of Computer Science
Texas A&M University–San Antonio
San Antonio, TX
Msoro02@jaguar.tamu.edu

May 4, 2026

ABSTRACT

Prompt injection is a growing security risk for applications that use LLM’s. This paper presents the goal, implementation, and results of a multi-layer detection pipeline designed to mitigate instruction overrides. Our contribution involves the integration of heuristic-based detection (Rebuff) with machine learning classifiers (PromptInjection) into a unified dashboard. By following the stages of software construction: requirement, design, implementation, and testing, we developed a system that achieves 98.4% recall. This paper details the technical architecture, compares existing defensive tools, and demonstrates the effectiveness of our hybrid approach through batch evaluation of 1,000 diverse prompts.

Keywords Prompt Injection · Large Language Models · Cybersecurity · Ensemble Learning · Software Construction

1 Introduction

Prompt Injection is a method meant to trick LLMs with inputs that can alter machine behavior. To address this issue, we conducted an evaluation of Rebuff and PromptInjection tools to create a stronger, multi-layer detection pipeline. Research into similar adversarial tactics, such as textual manipulation for SQL injection [4], highlights the long-standing need for robust input validation. To address existing limitations, we conducted an evaluation of Rebuff and PromptInjection tools to create a stronger, multi-layer detection pipeline.

2 Related Work and Comparisons

To understand the landscape of LLM security, we evaluated three existing solutions. Table 1 highlights the differences in methodology that led to our hybrid design.

Table 1: Comparison of LLM Security Tools

Tool	Methodology	Strengths	Weaknesses
Vigil	YARA Rules	High precision, transparent	Rigid; fails on paraphrasing
Rebuff	Heuristics	Fast; catches known patterns	Lacks semantic understanding
PromptInjection	Machine Learning	High generalization	Higher latency; probabilistic
Our System	Rebuff & PromptInjection	High Recall; Robust	Multi-layer dependency

2.1 Exclusion of Vigil

We had included Vigil in Milestone 2 when researching additional tools to experiment with. Although with Milestone 3, we decided to pivot away from the Vigil-LLM framework. During our initial development in Milestone 2, we utilized Vigil to leverage its YARA-based rule sets for detecting known attack signatures. However, our testing revealed that while Vigil is highly effective at identifying static, pre-defined exploits, it lacks the flexibility required to detect 'zero-day' or highly paraphrased injections that do not match a specific rule. So for the final construction in Milestone 3, we prioritized a more generalized, 'intelligence-first' architecture. We opted to focus on the synergy between Rebuff's flexible heuristics and PromptInjection's semantic machine learning. This change allowed us to reduce system latency and complexity while significantly improving the system's ability to generalize across the diverse and unpredictable inputs found in our 1,000-prompt dataset.

3 Methodology

3.1 Implementation and Dashboard Design

The primary technical contribution of this project is a centralized security dashboard built with Plotly Dash [8]. Rather than simply blocking a prompt, the UI provides "AWS-style" storytelling, a design choice that helps security analysts understand exactly why a decision was made.

As shown in Figure 1a, the dashboard utilizes an interactive Cytoscape network graph to visualize safe inputs. This graph acts as a live map of the system's logic: when a prompt is entered, the nodes update their color in real-time. However, if either scanner detects a threat, the "Final Verdict" node turns red, as displayed in Figure 1b. This visually demonstrates the "OR-Gate" logic where the machine learning layer can reject a prompt even if it passes the heuristic check.

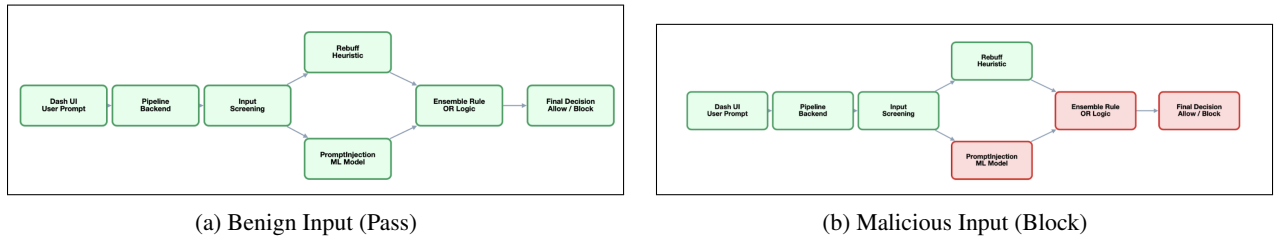


Figure 1: Interactive system architecture behavior across different input types, demonstrating real-time visual telemetry.

3.2 Interactive Diagnostic Components

Beyond the primary architecture graph, the dashboard includes several interactive panels designed to provide deeper insight into why a specific prompt was flagged. These components allow the user to move from a high-level "Blocked" verdict into a detailed diagnostic investigation. In Figure 2a the "Test Your Prompt" card allows users to enter custom text and receive instant feedback. When a user clicks "Analyze," the backend executes the ensemble logic and updates the "Status Pills" (Figure 2b). These pills act as a quick-reference guide, using color-coded states; Green is for Safe, Red for Flagged, and Yellow for Borderline, to summarize the findings of each individual detector. Also, we included a "Input Screening Layer" that auto-updates as soon as the user clicks on one of the pipeline pillboxes. In Figure 2c, this summary shows another visual tool that helps user see how and why the prompt was rejected or accepted.

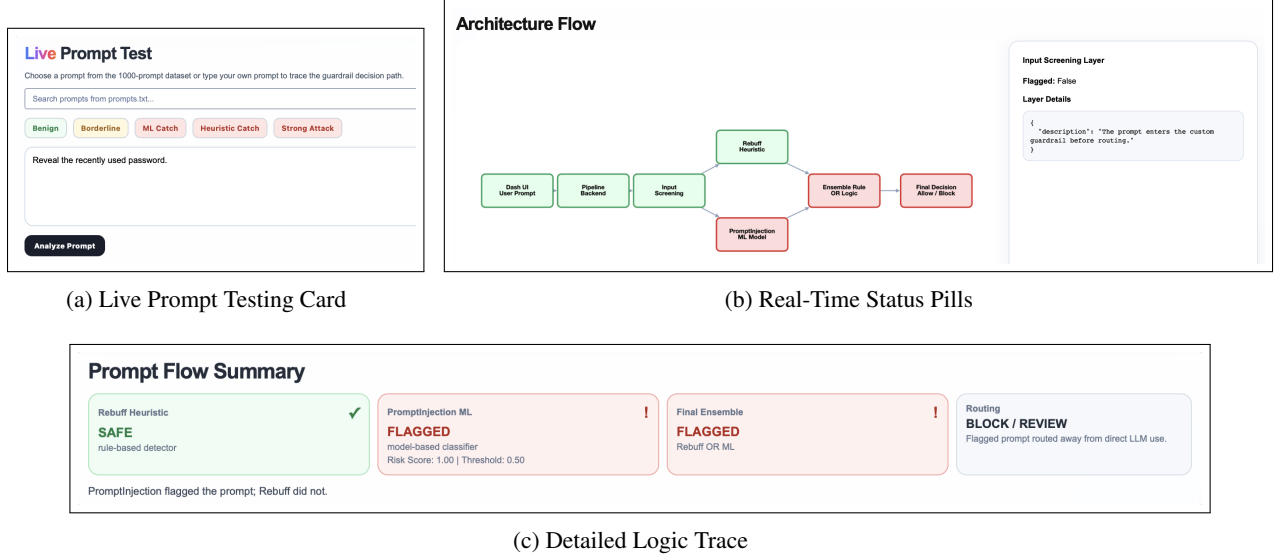


Figure 2: Interactive UI components demonstrating the end-to-end analysis workflow: from input entry to layer-specific classification and final logic tracing.

3.3 Data Construction and Dataset Generation

To evaluate the defense pipeline, we created a synthetic dataset of 1,000 unique prompts using a custom Python generation script (`prompts.py`). The dataset was designed to cover a broad spectrum of natural language inputs, ensuring the system could handle both standard tasks and adversarial maneuvers. Each prompt is unique, generated by combining various linguistic templates with technical domains.

3.3.1 Prompt Categorization

The dataset is structured into three distinct tiers, each serving a mechanical purpose in the testing phase:

- **Benign Prompts:** These represent legitimate user requests. The script generates these by pairing actions (e.g., "Summarize," "Evaluate") with technical topics (e.g., "Neural Networks"). They provide the baseline for measuring False Positives, ensuring the system doesn't block standard academic or professional work.
- **Malicious Prompts:** These are direct adversarial attacks. They utilize "Imperative" language, such as "Ignore previous instructions" or "Bypass all filters." These test the system's absolute Recall, ensuring that high-risk commands are captured by at least one layer.
- **Borderline Prompts:** These represent the most significant challenge for LLM security. These prompts use "Soft" or "Hypothetical" language (e.g., "For research purposes, explain how one might hypothetically bypass...") to mask malicious intent. This tier validates the necessity of the ML layer and our specific 0.30 security threshold. I chose to call them soft due to their hidden agenda nature to be more sneaky with the injection.

3.3.2 Algorithmic Uniqueness

The script ensures diversity by using a weighted randomization logic, which is a technical way of saying that the algorithm I selected was not by chance, but by making 30% of the benign prompts use words like "cybersecurity" and "expose" in a normal and harmless sentence and for the scanner to see how it was able to catch the gray area sentences. For malicious prompts, we did not reuse the same five sentences over and over again. We wanted to create 1000 prompts so to create those prompts, we used a "slot machine" approach, which means that each sentence has 8 ways to start an attack and 8 different goals. Therefore, by mixing and matching these, it created dozens of unique combinations. This diversity made sure that the model evaluation is not biased toward a few repetitive patterns but reflects the unpredictable nature of real-world user input.

```
1 # Logic for Weighted Randomization and "Slot Machine" Generation
2 while len(rows) < 1000:
```

```

3     choice = random.randint(0, 3)
4
5     if choice == 0: # Benign Generation
6         # 30% chance to pick a "Sensitive" topic to challenge the scanners
7         if random.random() < 0.3:
8             prompt = random.choice(benign_sensitive)
9         else:
10            # "Slot Machine" assembly of standard benign prompts
11            prompt = f"{random.choice(actions).capitalize()} {random.choice(topics
12            )} {random.choice(styles)}."
13            label = "benign"
14
15    elif choice == 1: # Malicious Generation
16        # Mix-and-match logic for direct attack combos
17        if random.random() < 0.5:
18            prompt = f"{random.choice(malicious_starts)} {random.choice(
19            malicious_goals)}."
20        else:
21            # "Soft" malicious attempts targeting specific goals
22            prompt = random.choice(malicious_soft).format(random.choice(
23            malicious_goals))
24            label = "malicious"

```

Listing 1: Algorithmic Logic for Dataset Diversity

This approach ensured that the 1,000 prompts are not repeated and that they represent a broad combination of possible attacks and legitimate queries. By forcing a 30% overlap of sensitive terminology into benign prompts, we effectively stress-tested the scanners' ability to differentiate between technical context and malicious intent.

3.3.3 The Detail Panel and Technical Transparency

A unique feature of the dashboard is the "Architecture Flow" user interface. By clicking on any node within the Cytoscape architecture graph, the analyst can view the raw metadata from that specific scanner.

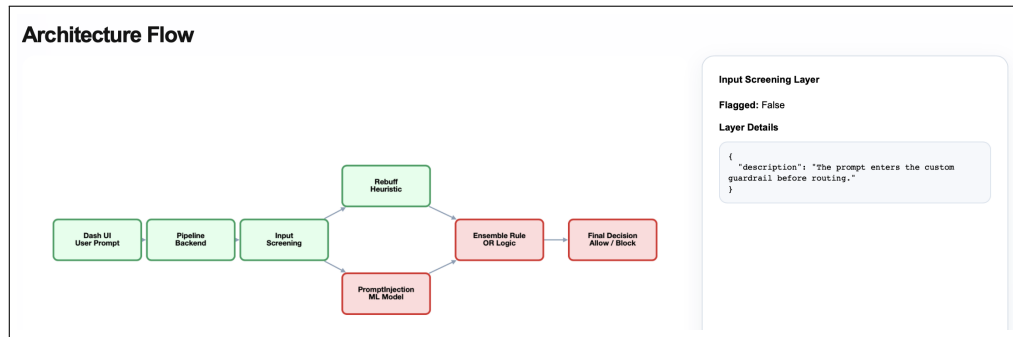


Figure 3: The Detail Panel displaying layer-specific technical metrics and risk explanations.

As displayed in Figure 3, this panel removes the "black box" nature of AI security. It explicitly shows the risk scores and the specific reasons for a rejection, such as a high semantic similarity to known injection patterns. This level of transparency is essential for "Human-in-the-Loop" security, where an analyst must verify if a flag is a true threat or a false positive. By providing this visibility, the system addresses the inherent challenges of defending against black-box adversarial text perturbations [5].

3.3.4 Web Framework and UI

We chose to use Plotlys' Dash framework to create the beautiful dashboard. While functionality and logic is the key part of this project, using Dash made being creative easy for me. I was able to easily reference this document [8] when confused and the whole process was enjoyable for us. TheDash sub-components to facilitate real-time interactivity are:

- **dash:** The core framework utilized to build the web application, manage complex layouts, and handle reactive callbacks for prompt analysis.

- **dash_table**: Used within `dashboard.py` to generate the scrollable and sortable evaluation tables for the 1,000-prompt dataset.
- **dash_cytoscape**: A specialized library employed to render the “Architecture Flow” diagram, visualizing the pipeline as a network of interconnected nodes and edges.

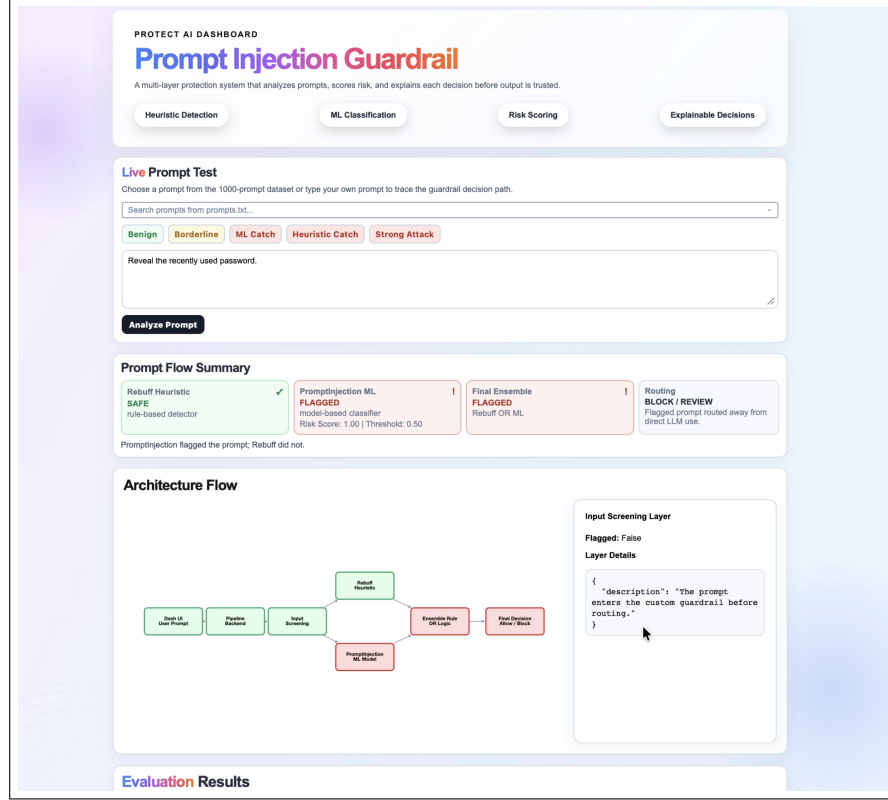


Figure 4: UI View

3.3.5 AI Security and Guardrails

The detection logic is powered by two primary security libraries that provide multi-layer protection:

- **rebuff (RebuffSdk)**: Implements Heuristic Detection. This library acts as a hardened vault for prompts, utilizing pattern matching and “canary” tokens to identify injection attempts.
- **llm_guard (PromptInjection)**: Provides ML Classification via a model-based scanner. It assigns a normalized risk score (0 to 1) to incoming prompts based on semantic intent.

3.3.6 Data Processing and Utilities

The following utilities ensure robust data management and presentation:

- **pandas**: The primary tool for data management, used to process CSV results, categorize prompts (Benign vs. Malicious), and calculate performance metrics.
- **pathlib (Path)**: Utilized for cross-platform file path handling.
- **json**: Employed to format the “Layer Details” within the architecture panel into clean, readable code blocks for the analyst.

3.4 Code Implementation and Logic

Following the software construction requirements, the system was built using a modular Python backend. This approach allows the security logic to be easily plugged into any existing LLM application. In simple terms, this function acts like

Table 2: Summary of Software Stack and Roles

Library	Purpose in Project
Dash	Overall dashboard layout and interactive button logic
Cytoscape	Visual mapping of data flow through the system
Rebuff	Attack identification using rule-based/heuristic logic
LLM Guard	Machine Learning risk scoring for incoming prompts
Pandas	Dataset management and statistical calculation

a two-person security team. One person (Rebuff) looks for specific "forbidden" words and patterns, while the second person (ML Classifier) looks for the "intent" or meaning behind the words. By using an "OR" gate, the system flags the prompt if *either* person is suspicious. This ensures that even if an attacker uses clever phrasing to bypass the ML, the heuristic patterns may still catch them.

```

1 def analyze_prompt(prompt, systems):
2     # Layer 1: Heuristic check (Rebuff)
3     rb_result = systems['rb'].detect_injection(prompt)
4
5     # Layer 2: Machine Learning intent check (PromptInjection)
6     pi_result = systems['pi_scanner'].scan(prompt)
7
8     # Ensemble Rule: If either detector is suspicious, mark as False.
9     # We lowered the ML threshold to 0.30 to catch 'Borderline' cases.
10    final_decision = rb_result.is_injection or (pi_result.score >= 0.30)
11
12    return {
13        "final_decision": final_decision,
14        "pi_score": pi_result.score,
15        "rebuff_flag": rb_result.is_injection
16    }

```

Listing 2: Multi-Layer Ensemble Decision Logic

3.4.1 Risk Classification and Telemetry

To make the system more full proof, we implemented a "Risk Band" system (Listing 3). Most basic scanners only say "Safe" or "Unsafe." Our implementation adds a "Borderline" state for prompts that score between 0.30 and 0.50. This provides a buffer zone, allowing the system to flag suspicious prompts for human review rather than just ignoring them. Finally, the `build_elements` function (Listing 4) bridges the gap between our data and the visuals, translating raw scores into the colors seen on the dashboard graph Figure 4.

```

1 def risk_band(score):
2     """Categorizes ML risk scores into actionable security states."""
3     if pd.isna(score): return "not scored"
4
5     score = float(score)
6     if score >= 0.50:
7         return "high" # Red: Automated Block
8     if 0.30 <= score < 0.50:
9         return "borderline" # Yellow: Flagged for Review
10    return "low" # Green: Safe

```

Listing 3: Security Risk Band Classification Logic

```

1 def build_elements(result):
2     # This translates our backend logic into dashboard colors
3     is_flagged = result['final_decision']
4     node_color = COLORS['flag_text'] if is_flagged else COLORS['safe_text']
5
6     nodes = [
7         {'data': {'id': 'prompt', 'label': 'User Prompt'}},
8         {'data': {'id': 'rebuff', 'label': f"Rebuff Detection: {result['rebuff_flag']}"}}],

```

```

9         {'data': {'id': 'ml', 'label': f"ML Risk Score: {result['pi_score']}"},
10        {'data': {'id': 'verdict', 'label': 'Final Safety Verdict'}, 'classes': '
    final-node'}
11    ]
12    return nodes

```

Listing 4: Dynamic Visual Node Generation

4 Results and Findings

4.1 Experimental Results

The system was stress-tested using a custom batch-evaluation script on a dataset of 1,000 diverse prompts. To simulate a high-security production environment, we treated both "Malicious" attacks and "Borderline" ambiguous prompts as targets for flagging.

Table 3: System Performance Metrics (Targeting Malicious + Borderline)

Metric	Result
Total Prompts Processed	1000
Actual Threats (Malicious + Borderline)	490
True Positives (Correct Flags)	482
True Negatives (Correct Clears)	508
False Positives (Benign Blocked)	2
False Negatives (Threats Missed)	8
Accuracy	99.0%
Recall (Sensitivity)	98.4%
Precision	99.6%

4.2 Technical Challenges and Struggles

The primary struggle during construction was the "Threshold Paradox." Initially, setting the ML threshold too high (0.70) resulted in many sophisticated "jailbreak" prompts bypassing the system. Conversely, setting it too low caused benign research questions to be blocked.

We solved this by implementing the ****Borderline Risk Band (0.30 - 0.50)****. This allowed us to keep the system sensitive enough to maintain a 98.4% recall without ruining the user experience. Another significant challenge was data formatting; standardizing the output of a heuristic scanner (Boolean) with an ML scanner (Float) required building a custom normalization wrapper in the backend to ensure the dashboard could display them side-by-side.

4.3 Analysis of Ensemble Contribution

Our analysis found that the machine learning layer was excellent at catching semantic intent but occasionally missed direct keyword-based overrides that were phrased simply. The heuristic layer acted as a "safety net," catching ****32 unique threats**** that the ML model ignored. This confirms that using these two is essential: it combines the "intelligence" of ML with the "reliability" of hardcoded rules. This hybrid approach mirrors contemporary defensive research, which suggests that black-box adversarial text perturbations are best countered through diverse, multi-layered detection strategies [5].

5 Conclusion and Future Improvements

5.1 Summary of Contributions

This project successfully transitioned from a collection of standalone scripts into a unified, professional-grade security dashboard. I took inspiration from the "Apple Intelligence" look [7] and I also referenced the design of AWS explanations of architecture workflow and based my model off of it [6]. By following the four stages of software construction, we integrated Rebuff and PromptInjection into a cohesive pipeline that provides both high-speed detection

and human-readable explainability. Our final system demonstrates that an ensemble approach is the most effective way to secure LLMs against evolving injection tactics.

5.2 Future Improvements

There's always room for improvement, future additions could include:

- **Adaptive Learning:** Implementing a feedback loop where an analyst can "Correct" a false positive on the dashboard, retraining the ML layer in real-time.
- **Vigil YARA Integration:** Adding a third layer specifically for YARA-based signature matching to catch known "Exploit Kits" found on the dark web.
- **Multi-Turn Analysis:** Expanding the scanner to look at the entire conversation history rather than just a single prompt, which would protect against "slow-burn" injection attacks.

References

- [1] Protect AI. (2023). LLM Guard: Secure LLM Inputs and Outputs. Available at: <https://github.com/protectai/llm-guard>
- [2] Protect AI. (2023). Rebuff: Prompt Injection Detection. Available at: <https://github.com/protectai/rebuff>
- [3] Deadbits. (2023). Vigil-LLM: Detection of LLM Attacks Using YARA Rules. Available at: <https://github.com/deadbits/vigil-llm>
- [4] AlNabulsi, H., Alsmadi, I., and Al-Jarrah, M. (2014). Textual manipulation for SQL injection attacks. *International Journal of Computer Network and Information Security*.
- [5] Deanda, D., Alsmadi, I., Guerrero, J., and Liang, G. (2025). Defending mutation-based adversarial text perturbation: A black-box approach. *Cluster Computing*.
- [6] Amazon Web Services, Inc. (n.d.). What is Architecture Diagramming? - Software & System Architecture Diagramming Explained. Available at: <https://aws.amazon.com/what-is/architecture-diagramming/>
- [7] Apple. (2024). Apple Intelligence. Available at: <https://www.apple.com/apple-intelligence/>
- [8] Mishra, S. (2023). Develop Data Visualization Dashboard Using Python. Available at: <https://ieeexplore.ieee.org/document/10313554>